



Co-funded by
the European Union



SMASH
machine learning for science and humanities postdoctoral program

Data Analysis and Machine Learning Basics

Alex Razim, University of Nova Gorica, Slovenia

VISTULA workshop
Warsaw, Sep 22, 2026

Lecture outline

Part 0: Why are you here?

Part I: What is Machine Learning

- 'Behaviour from the data' principle
- Optimization problem. Loss and cost functions
- Gradient descent and backpropagation
- SLP and MLP
- Shallow and deep NNs
- ML interpretability problem
- ML types: supervised/un-/semi-/self-supervised
- Classification, regression, clusterization, prediction

Part III: Interrogating the black box

- Train/validation/test sets. Cross-validation
- Bias-variance tradeoff
- Overfitting
- Precision-recall, F1 score
- Confusion matrix, ROC, AUC
- Typical failure modes

Alex Razim, *astroinformatician* = astronomy + ML + Data science

Boring biography stuff

- ✓ I'm from Ukraine
- ✓ BSc & MSc: Processing Moon images
- ✓ PhD: Photometric redshifts (aka 'distances to many-many galaxies, but with dubious accuracy') with ML.
 - Defended during Covid!
 - Survived Naples!
- ✓ First postdoc (Zagreb, Croatia): developed a catalogue of synthetic LSST magnitudes, organized a bunch of Python workshops for astronomers.
- ✓ Second postdoc: SMASH fellow in the University of Nova Gorica, Slovenia

The dog to look at while I babble



Science: rare variability search

TL;RL: we need efficient novelty detection methods for PB-scale astronomica datasets. I look for these methods.

Example:

Take large astro-survey

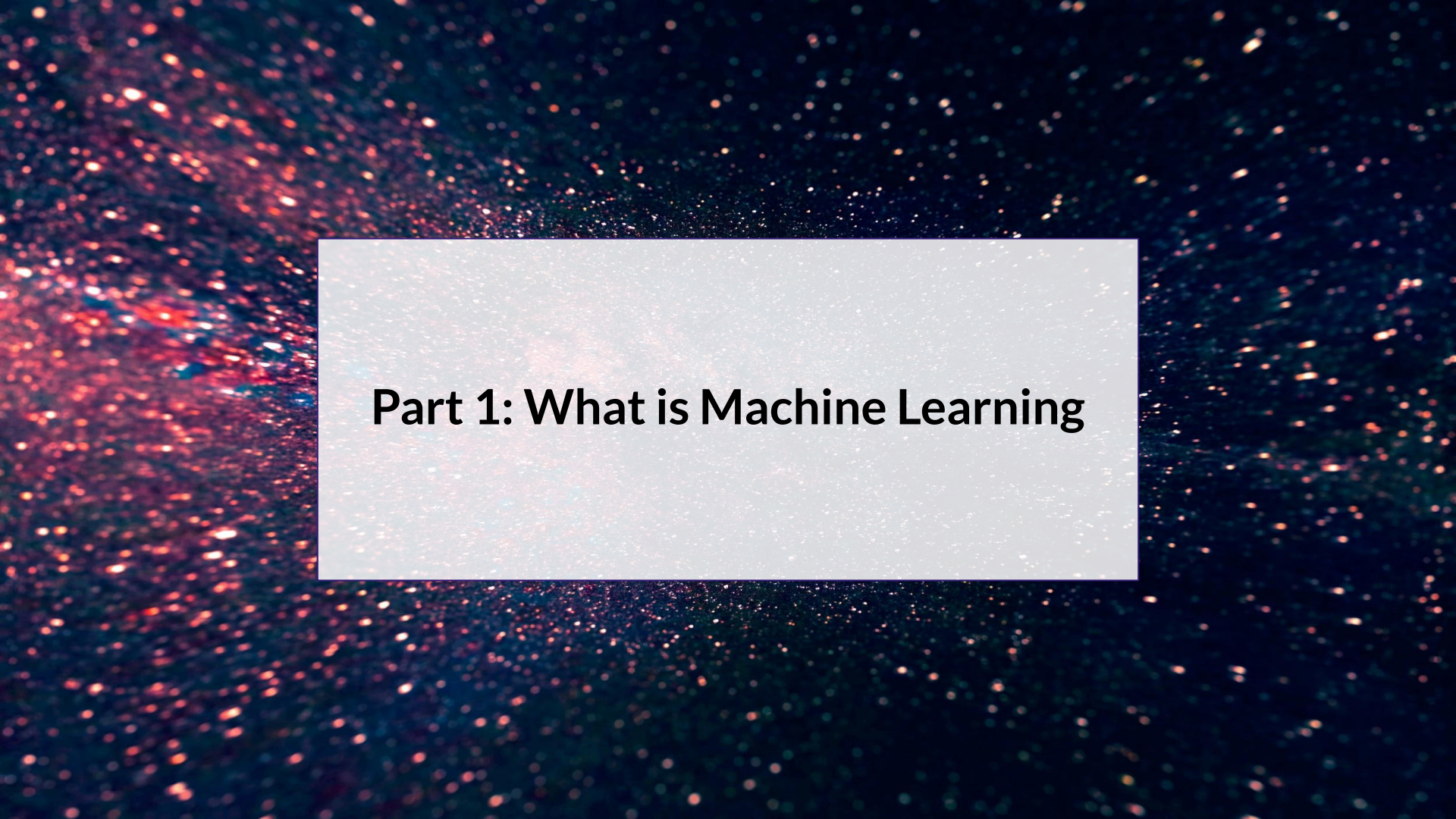
Extract features using
community-developed packages

Feed to a bunch of UML algorithms

See if we can find previous accidental
discoveries semi-automatically

**Part 0: Who are you people,
and why are you here?**

- ❑ What is your topic?
- ❑ What question your work tries to answer?
- ❑ How often do you write code?
- ❑ How often do you *generate* code?
- ❑ Which coding languages?
- ❑ Did you use ML before?
- ❑ Does your group uses ML?
- ❑ What should happen during this workshop for you to feel that you spent your time well?



Part 1: What is Machine Learning

Machine learning – algorithms that learn from data and can perform tasks for which they were not explicitly programmed.

But what do we mean by ‘learning’?..

Machine learning – algorithms that learn from data and can perform tasks for which they were not explicitly programmed.

But what do we mean by ‘learning’?..

ML method is a method whose capabilities improve as it sees more data (D. Hogg) ->

Machine learning – algorithms that learn from data and can perform tasks for which they were not explicitly programmed.

But what do we mean by ‘learning’?..

ML method is a method whose capabilities improve as it sees more data (D. Hogg) ->

Learning is the ability to perform the task better when presented with more (relevant) data.

ML as advanced calculus

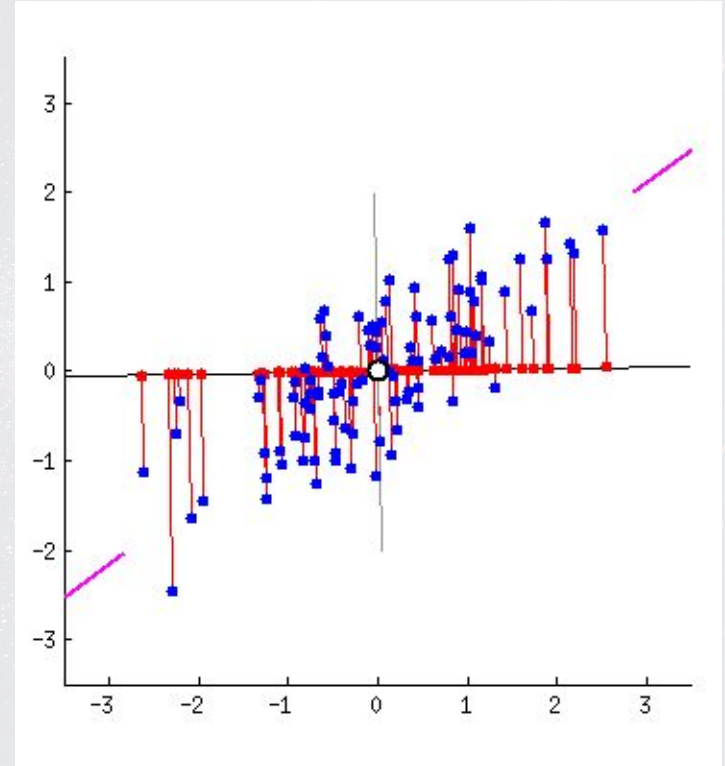
Machine learning – algorithms that learn from data and can perform tasks for which they were not explicitly programmed.

But what do we mean by ‘learning’?..

ML method is a method whose capabilities improve as it sees more data (D. Hogg) ->

Learning is the ability to perform the task better when presented with more (relevant) data.

From the practical point of view,
learning = optimization = minimizing (or maximizing) certain target value.



Minimizing what?

...sorry, terminology detour first.

Minimizing what?

...sorry, terminology detour first.

Data point (sometimes called 'sample') - a set of observed values ('features') describing an object or an event (in practice: rows in the table).

Label (or 'target') - the property of the data points that we try to learn, e.g. a classification ('elliptical or spiral', 'a star or a quasar') or a numerical value of interest (e.g. a sersic index or a redshift). In practice: the columns.

Prediction - the label that the trained model assigns to the data point.

(coming back) So, minimizing what?

Loss function - how we calculate an error for a single prediction. E.g.:

$$\text{square_loss} = (y_{\text{pred}} - y_{\text{true}})^2$$

Cost function - loss function, averaged across the whole sample. E.g.:

Mean Squared Error, aka

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_{\text{pred}}^i - y_{\text{true}}^i)^2$$

How MSE would behave if you have extreme outliers?..

Minimizing what?

Loss function - how we calculate an error for a single prediction. E.g.:

$$\text{square_loss} = (y_{\text{pred}} - y_{\text{true}})^2$$

Cost function - loss function, averaged across the whole sample. E.g.:

Mean Squared Error, aka

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_{\text{pred}}^i - y_{\text{true}}^i)^2$$

How MSE would behave if you have extreme outliers?..

MSE penalizes extreme outliers strongly. An alternative - Median Absolute Error (MAD):

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_{\text{pred}}^i - y_{\text{true}}^i|$$

Minimizing what?

Loss function - how we calculate an error for a single prediction. E.g.:

$$\text{square_loss} = (y_{\text{pred}} - y_{\text{true}})^2$$

Cost function - loss function, averaged across the whole sample. E.g.:

Mean Squared Error, aka

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_{\text{pred}}^i - y_{\text{true}}^i)^2$$

How MSE would behave if you have extreme outliers?..

MSE penalizes extreme outliers strongly. An alternative - Median Absolute Error (MAD):

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_{\text{pred}}^i - y_{\text{true}}^i|$$

There is no universal cost (or loss) function. You have to choose one depending on your task (or sometimes design one)

Gradient descent

We want to minimize the value of the loss function L ->
we have to move in the direction of negative gradient ->
change inputs of L to move it in the direction of negative gradient ->
gradient descent

$$w_{new} = w_{old} - \eta \frac{dL}{dw_{old}}$$

where L is the loss function, and η learning rate

Linear regression example

$$y = wx + b$$

$$L(w, b) = \frac{1}{N} \sum_{i=1}^N ((wx_i + b) - y_i^{true})^2$$

x - our data - is fixed, y^{true} - our labels - are fixed. We can change only w and b
For w ,

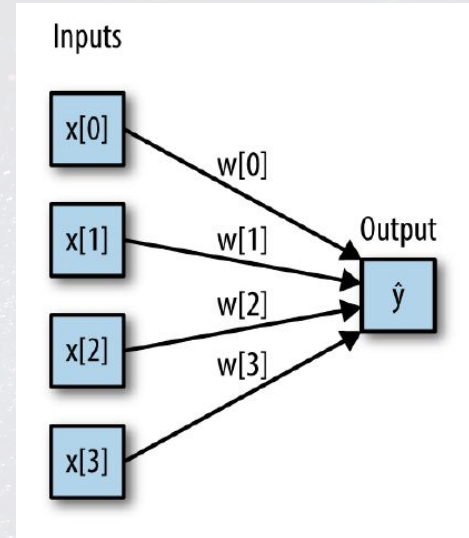
$$\begin{aligned} \frac{dL}{dw_{old}} &= \frac{dL}{dy_{old}} \cdot \frac{dy_{old}}{dw_{old}} = \\ &= \frac{d((y_{old} - y_{true})^2)}{dy_{old}} \cdot \frac{d(w_{old}x + b)}{dw_{old}} = \\ &= 2(y_{old} - y_{true}) \cdot x \end{aligned}$$

(pay attention, we have to do the same for b as well)

How to approximate anything

1. Single feature -> many features:

$$\hat{y} = w[0] * x[0] + w[1] * x[1] + ... + w[p] * x[p] + b$$

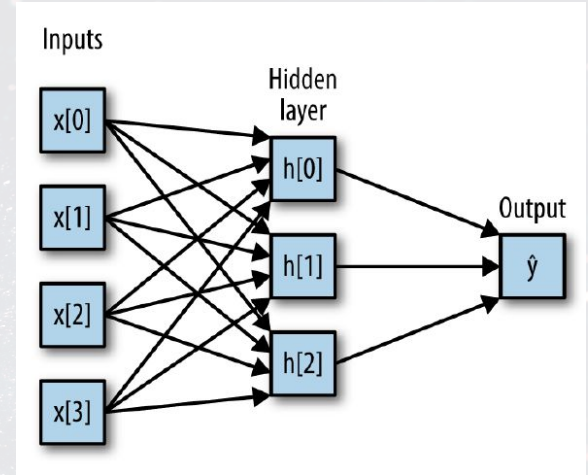


How to approximate anything

1. Single feature -> many features:

$$\hat{y} = w[0] * x[0] + w[1] * x[1] + \dots + w[p] * x[p] + b$$

2. Add layers



How to approximate anything

1. Single feature -> many features:

$$\hat{y} = w[0] * x[0] + w[1] * x[1] + ... + w[p] * x[p] + b$$

2. Add layers

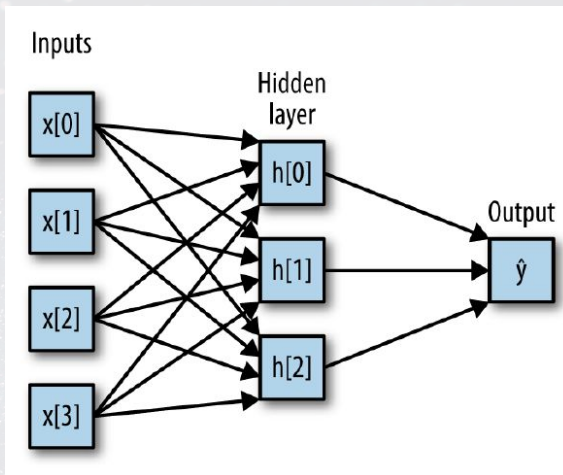
3. Add an **activation function** (because without one you always produce only straight lines)

$$h[0] = \tanh(w[0, 0] * x[0] + w[1, 0] * x[1] + w[2, 0] * x[2] + w[3, 0] * x[3])$$

$$h[1] = \tanh(w[0, 1] * x[0] + w[1, 1] * x[1] + w[2, 1] * x[2] + w[3, 1] * x[3])$$

$$h[2] = \tanh(w[0, 2] * x[0] + w[1, 2] * x[1] + w[2, 2] * x[2] + w[3, 2] * x[3])$$

$$\hat{y} = v[0] * h[0] + v[1] * h[1] + v[2] * h[2]$$



But how do we update all the weights?..

Que **backpropagation**:

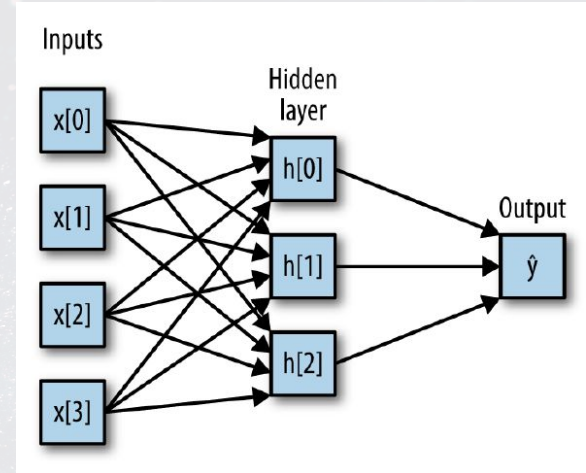
$$y_{pred} = w_2 \cdot h + b_2;$$

$$h = \text{tahn}(z);$$

$$z = w_1 \cdot x + b_1$$

$$\frac{dL}{dw_1} = \frac{dL}{dy_{pred}} \cdot \frac{dy_{pred}}{dw_1} =$$

$$\frac{dL}{dy_{pred}} \cdot \frac{dy_{pred}}{dh} \cdot \frac{dh}{dz} \cdot \frac{dz}{dw_1}$$



But how do we update all the weights?..

Que **backpropagation**:

$$y_{pred} = w_2 \cdot h + b_2;$$

$$h = \text{tahn}(z);$$

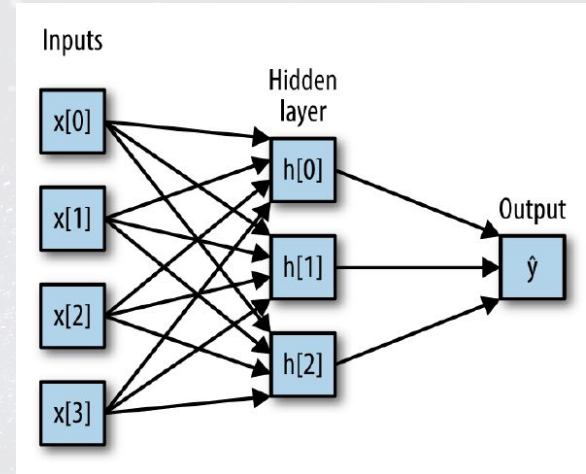
$$z = w_1 \cdot x + b_1$$

$$\frac{dL}{dw_1} = \frac{dL}{dy_{pred}} \cdot \frac{dy_{pred}}{dw_1} =$$

$$\boxed{\frac{dL}{dy_{pred}}} \cdot \boxed{\frac{dy_{pred}}{dh} \cdot \frac{dh}{dz}} \cdot \boxed{\frac{dz}{dw_1}} \quad \leftarrow x$$

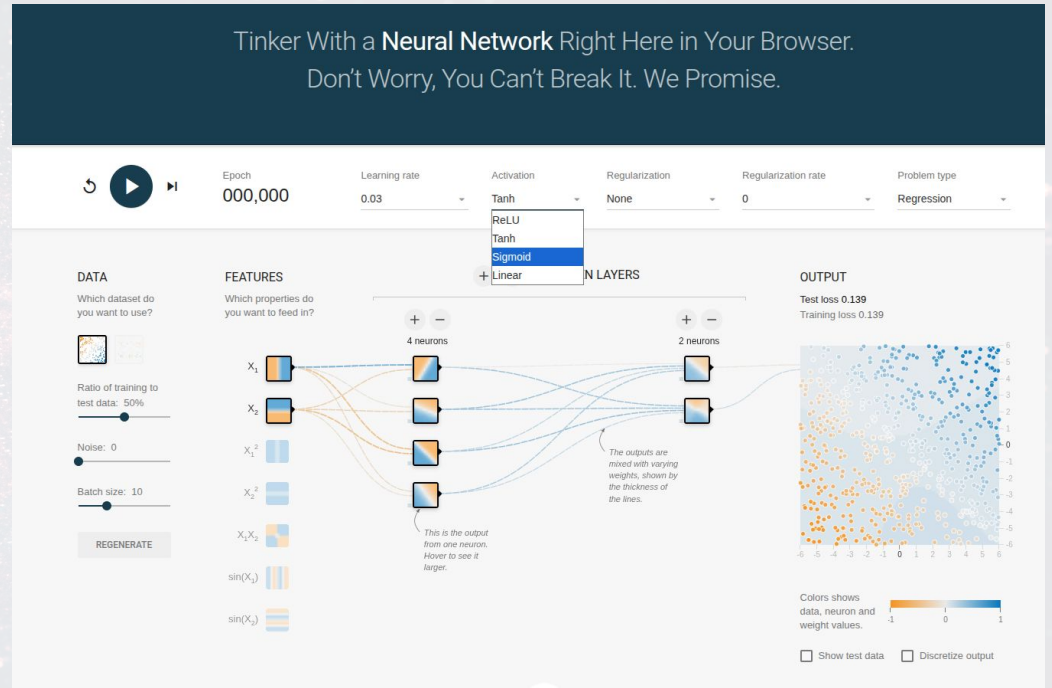
backpropagated
derivative

local derivative for the
neuron



Playtime

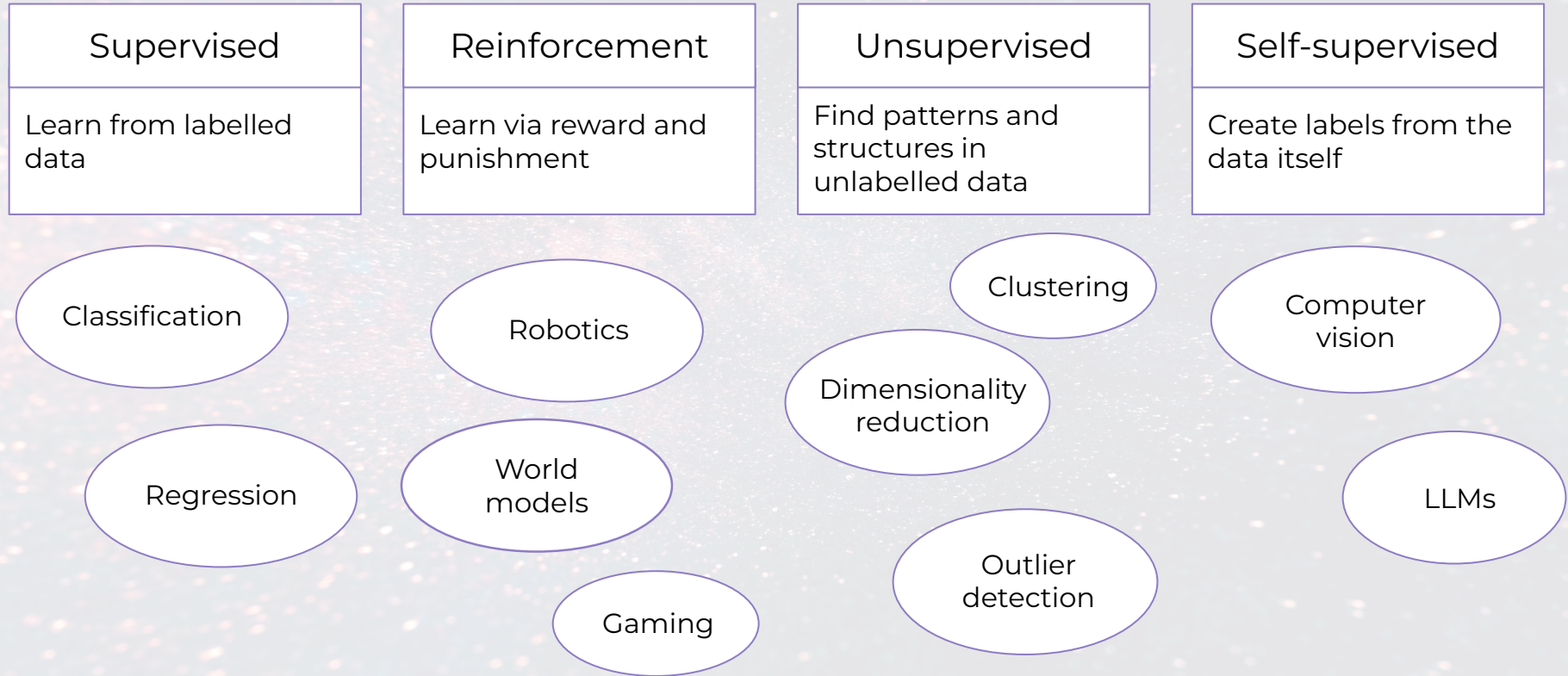
<https://playground.tensorflow.org/>



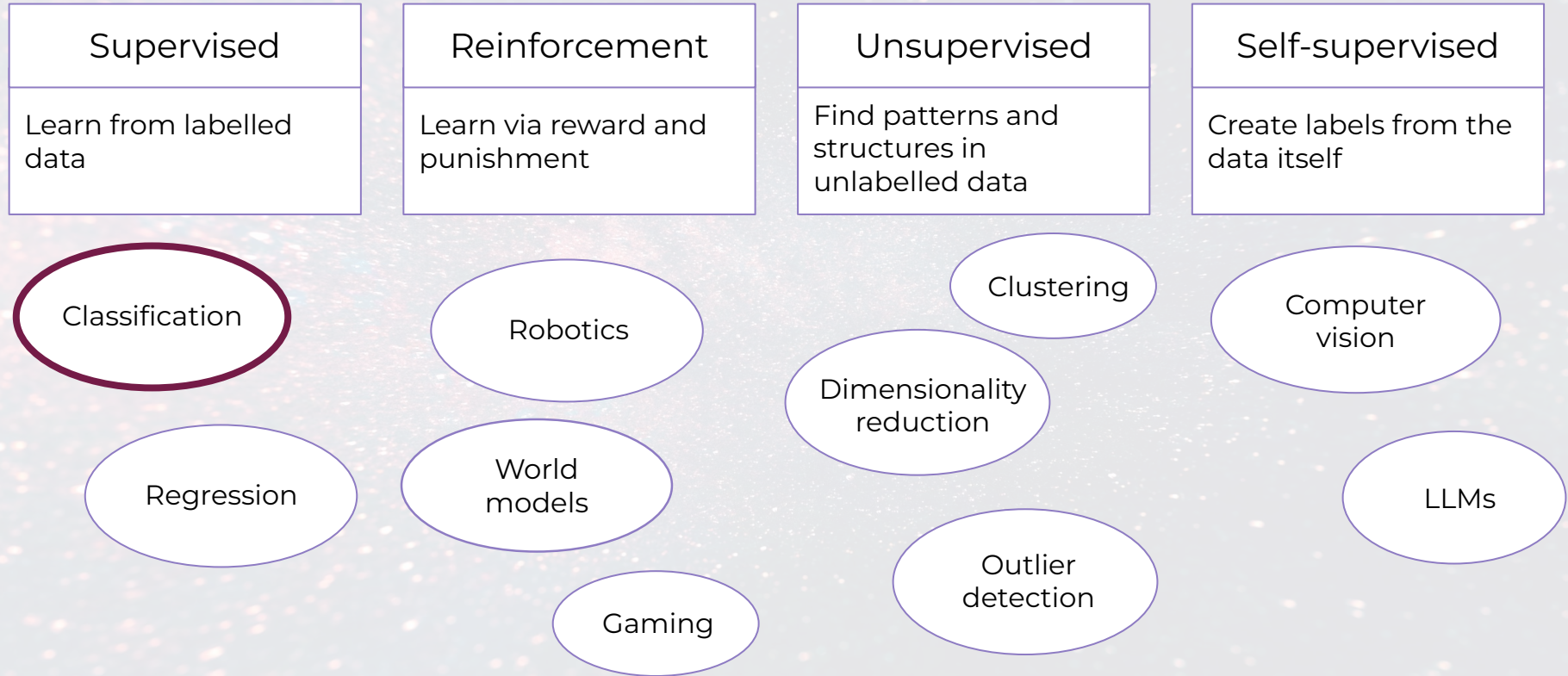


Part 2: ML Zoo

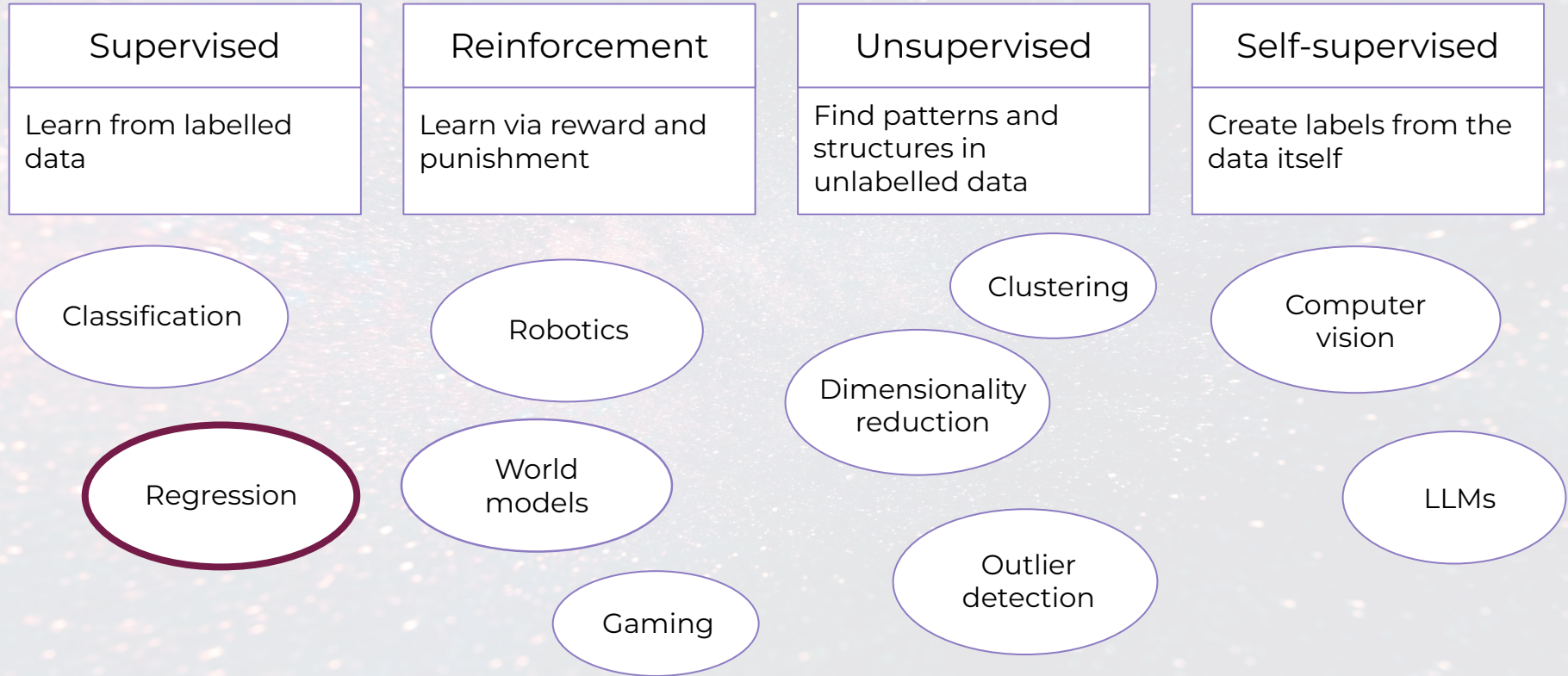
ML taxonomy



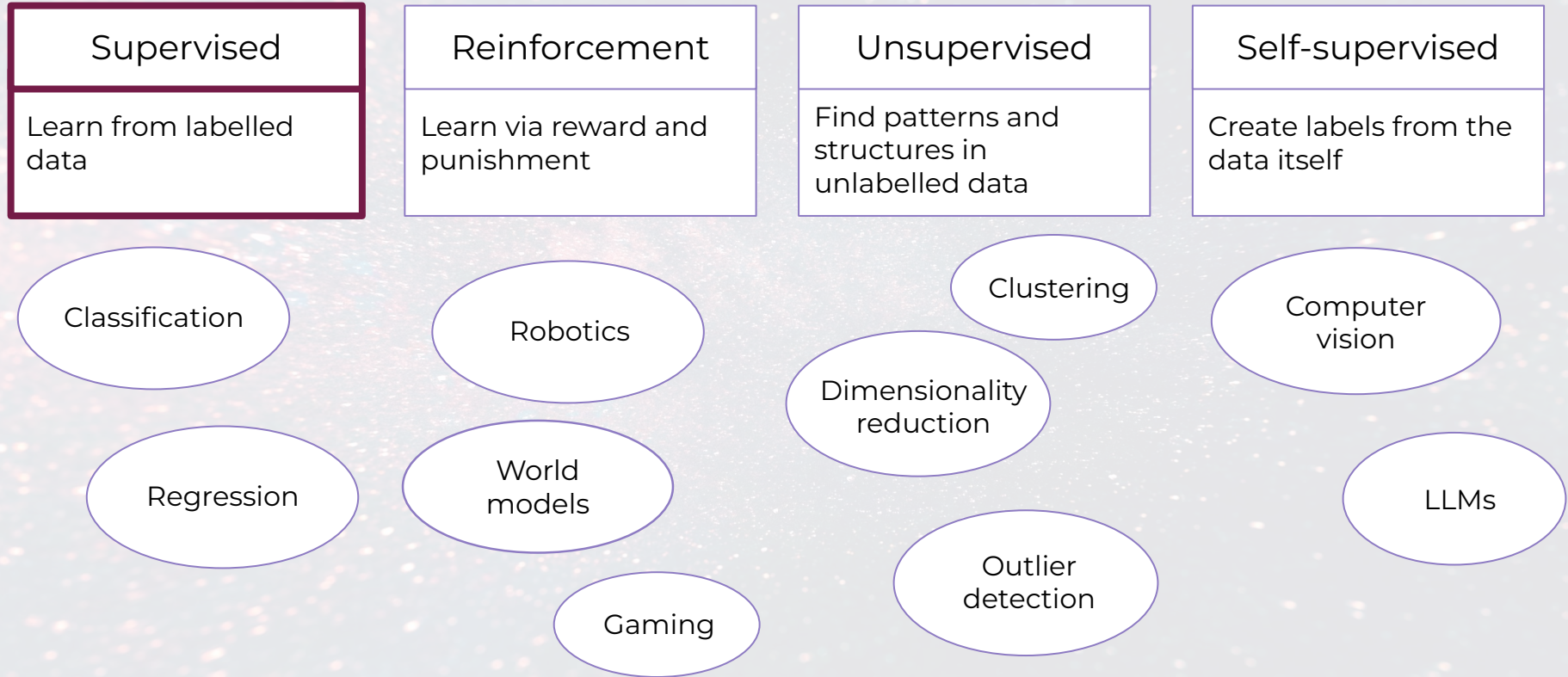
ML taxonomy



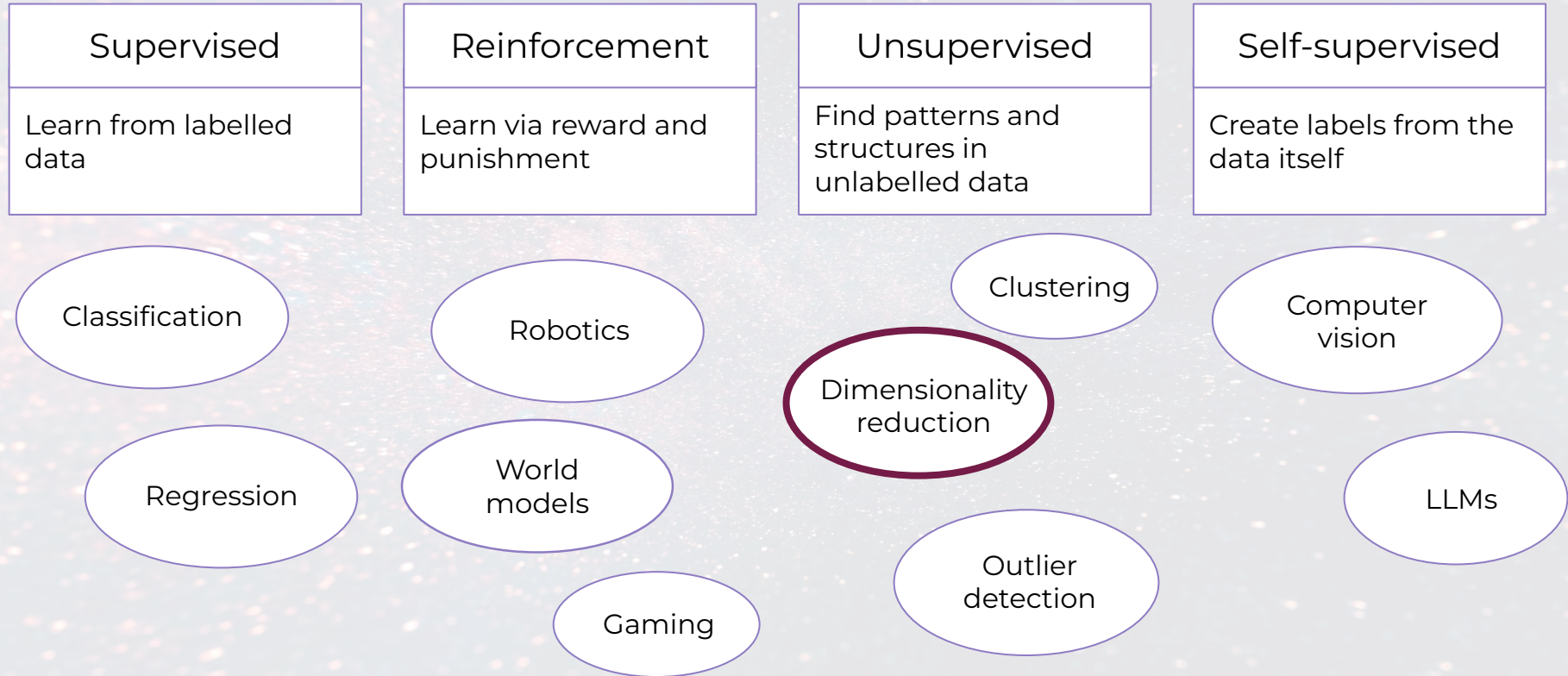
ML taxonomy



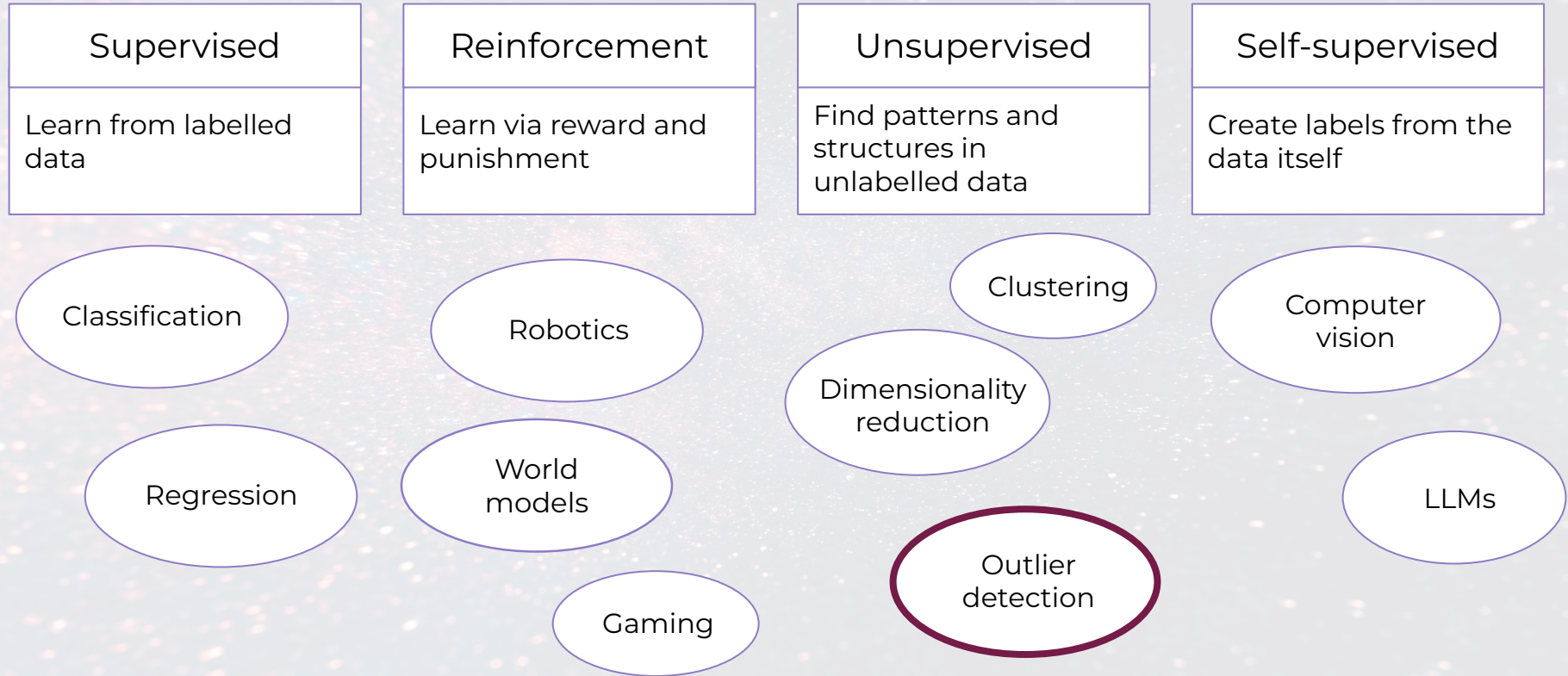
ML taxonomy



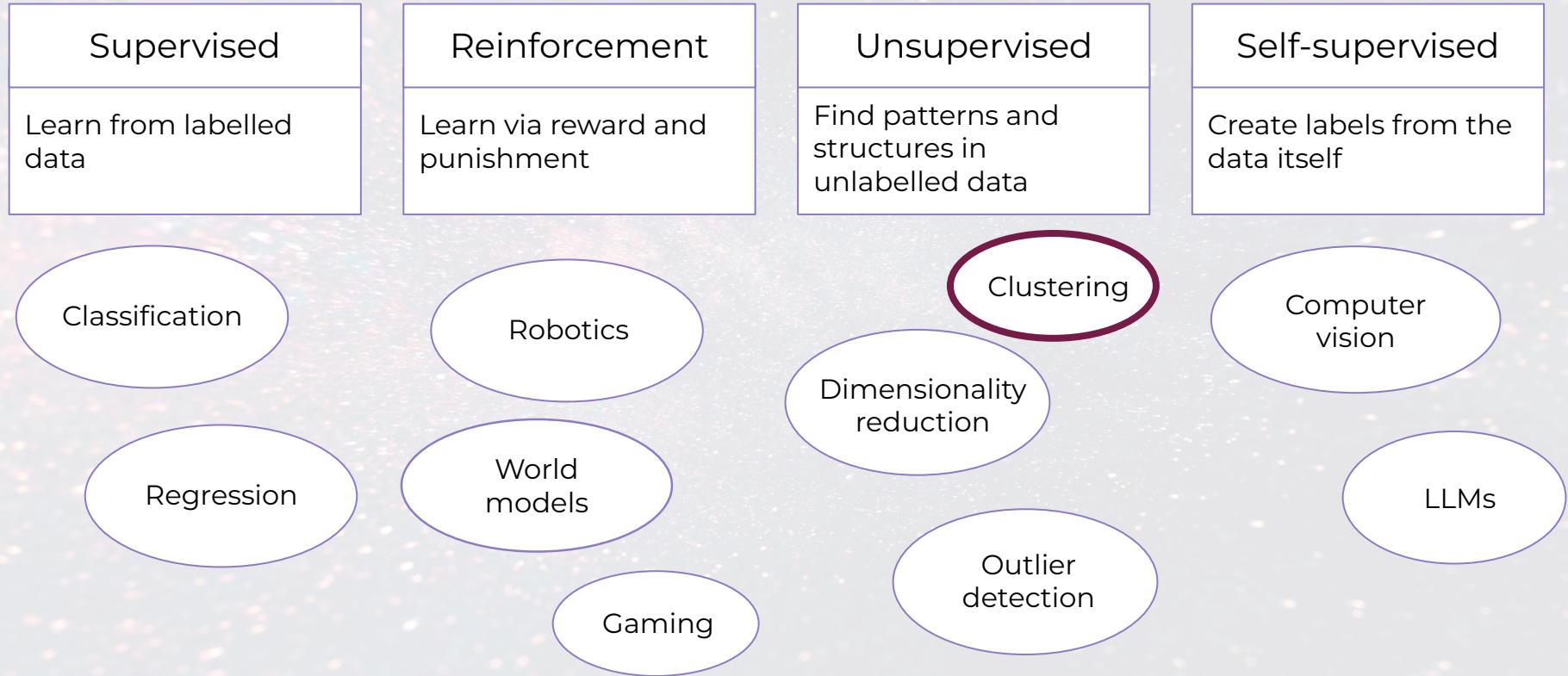
ML taxonomy



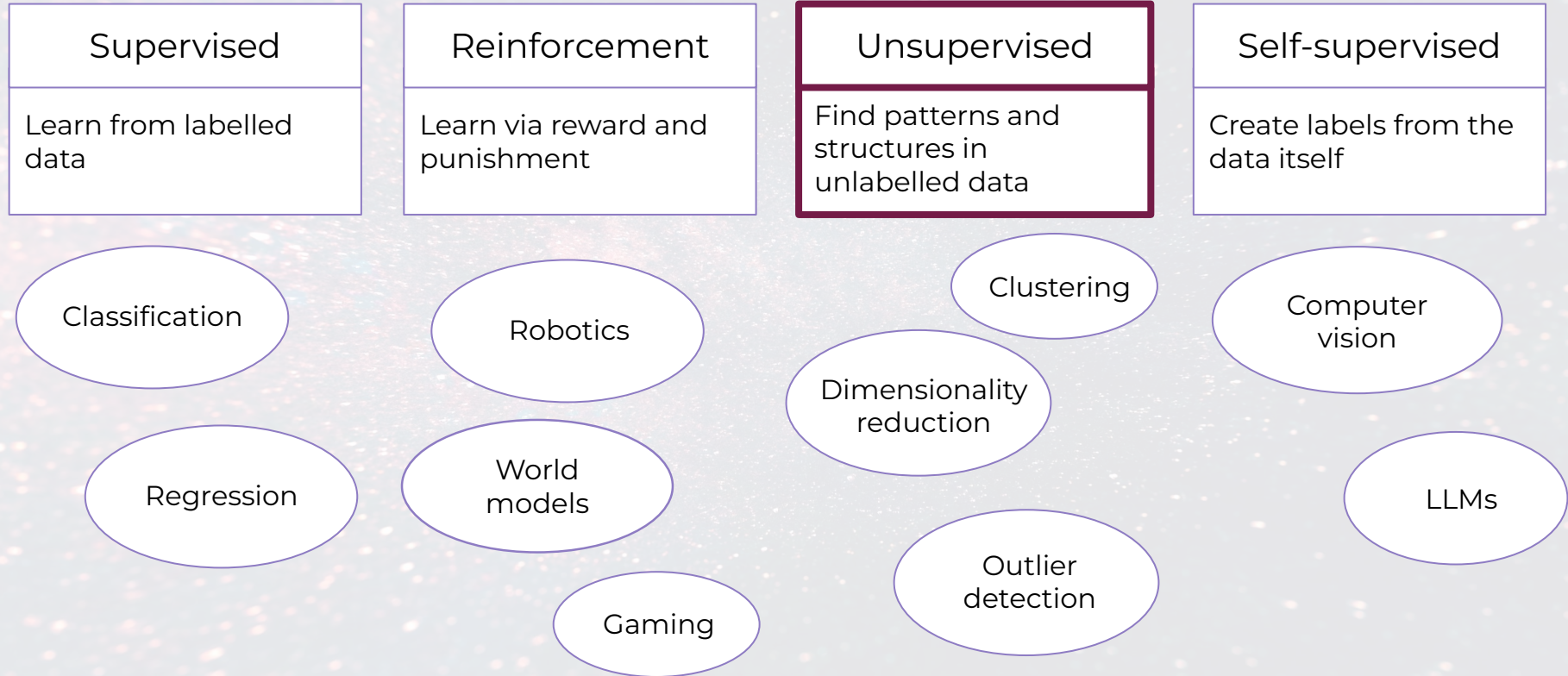
ML taxonomy



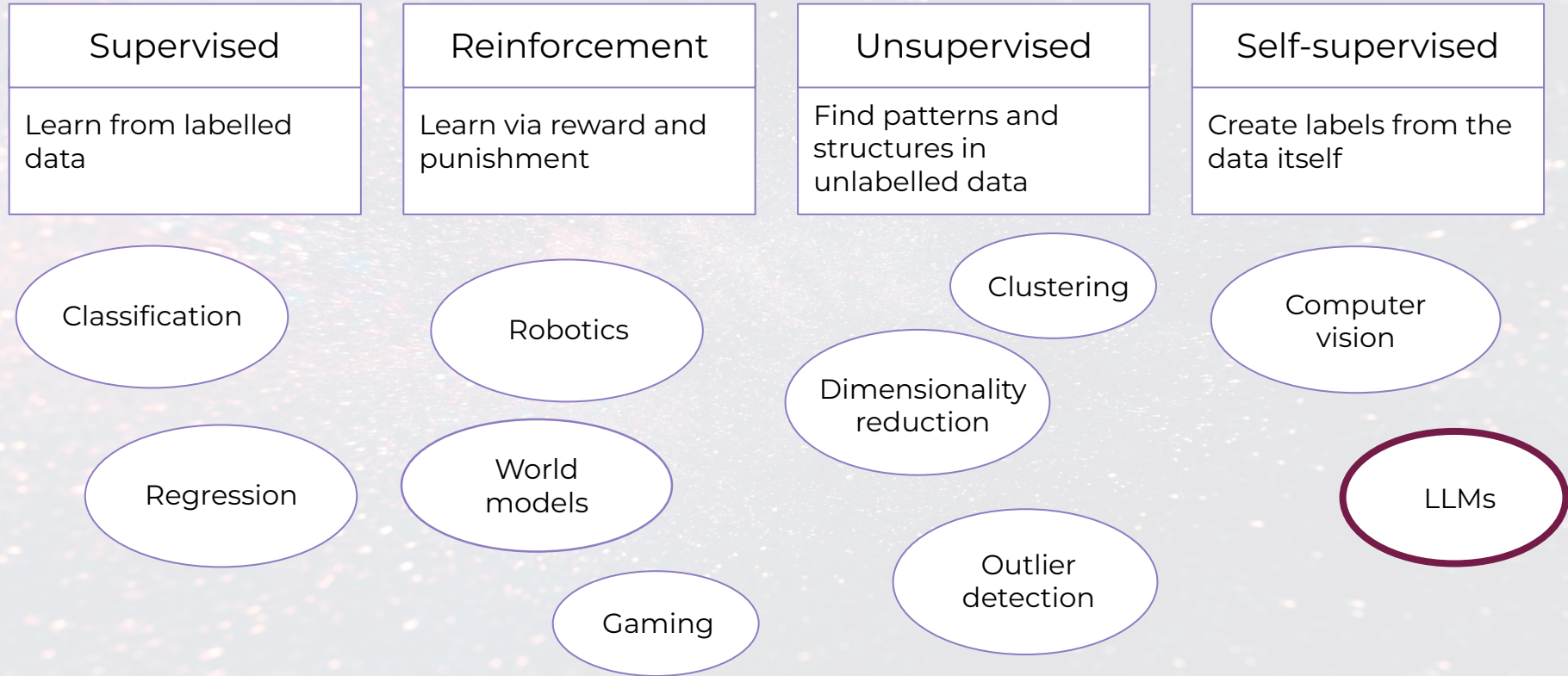
ML taxonomy



ML taxonomy



ML taxonomy





Part 3: Interrogating the Black Box

Exploratory Data Analysis or EDA

Exploratory Data Analysis = systematic inspection of structure, distributions, relationships and anomalies in the dataset.

Purpose: detect errors, form hypotheses, and understand modeling constraints.

Steps:

- ❑ Dataset shape - size and dimensionality
- ❑ Missing value analysis
- ❑ Summary statistics
- ❑ Feature distributions
- ❑ 2D relationships
- ❑ Correlation analysis
- ❑ Target variable inspection
- ❑ Lots of plots. LOTS.

'Five number summary':

- Extrema: min, max
- Median
- Q1, Q3 (values that split the lowest 25% of data from the highest 75% and vice versa) -> Q3-Q1 aka 'interquantile range' is a common ML feature!

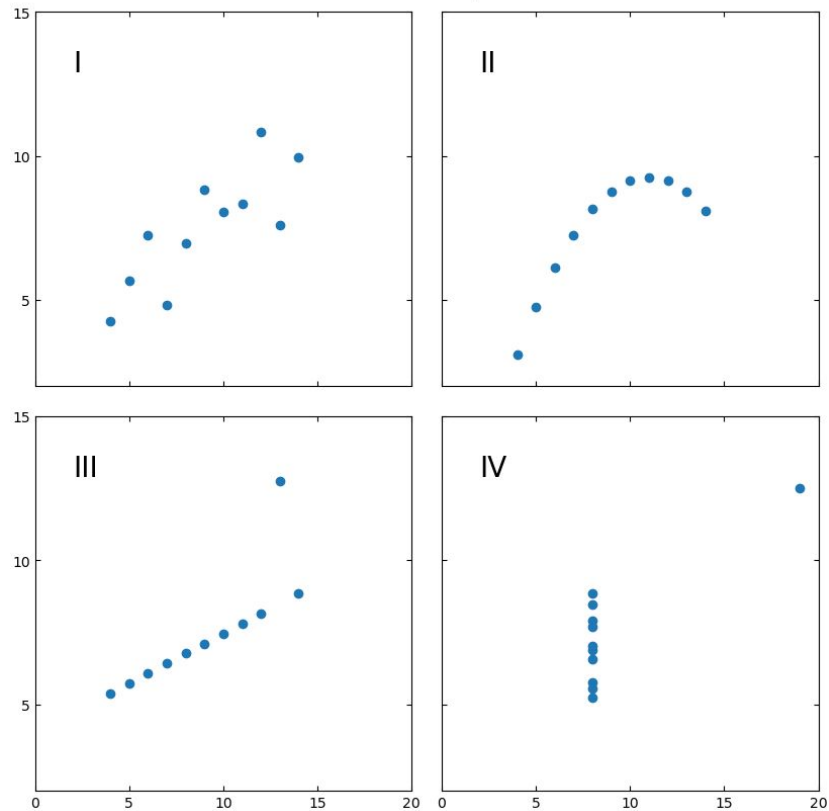
Also:

- Mean (but median is more robust)
- Standard deviation (but quantiles are more robust again)

EDA: Lies, damned lies, and statistics

Which statistical indicators you'd use to describe these datasets in a paper?

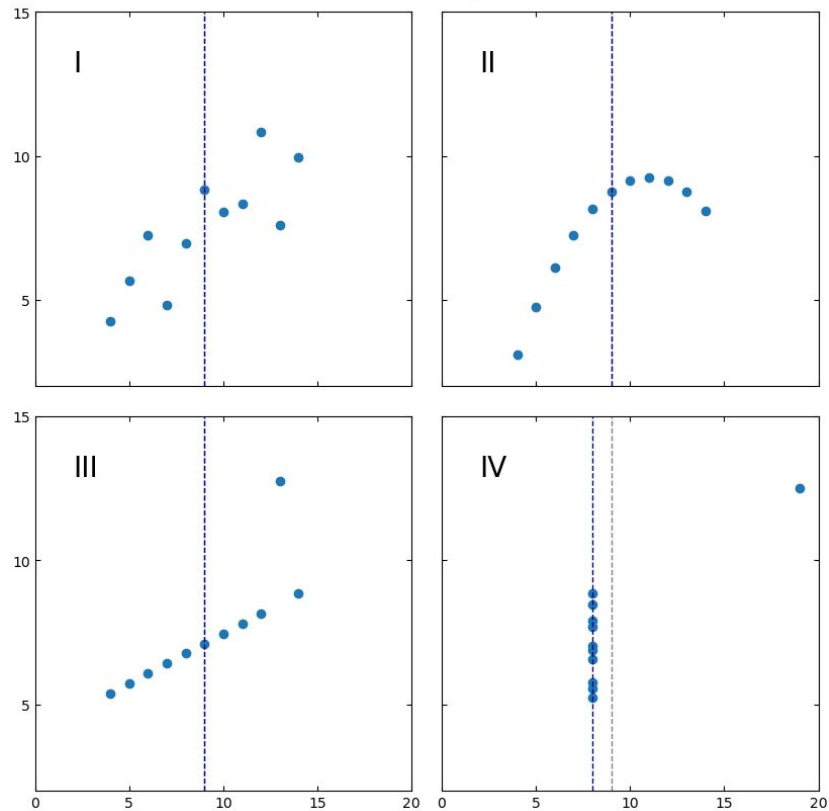
Anscombe's quartet



EDA: Lies, damned lies, and statistics

Which statistical indicators you'd use to describe these datasets in a paper?

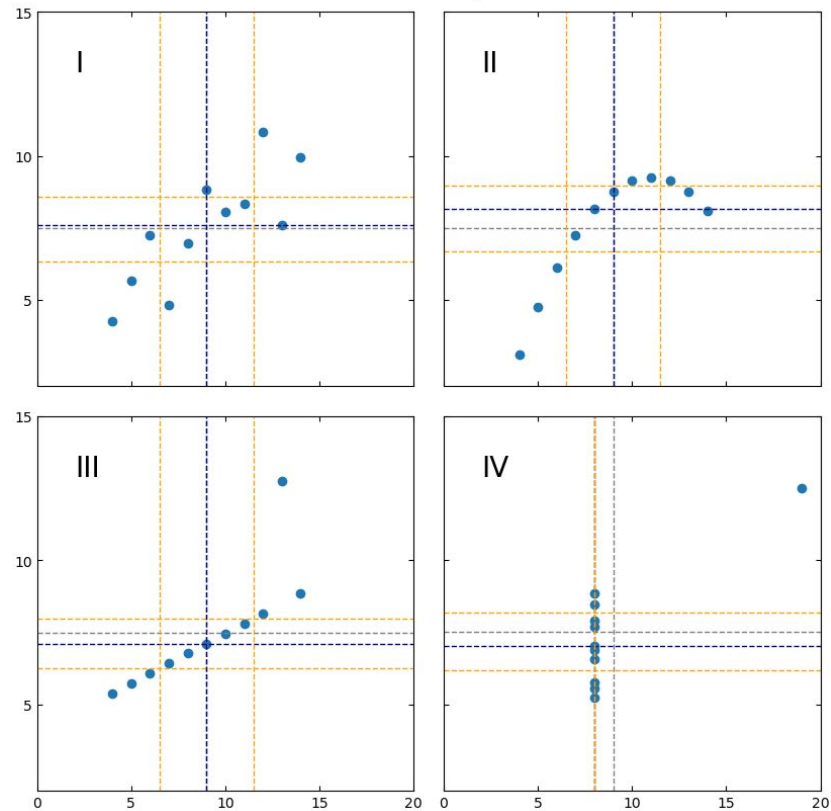
Anscombe's quartet



EDA: Lies, damned lies, and statistics

Which statistical indicators you'd use to describe these datasets in a paper?

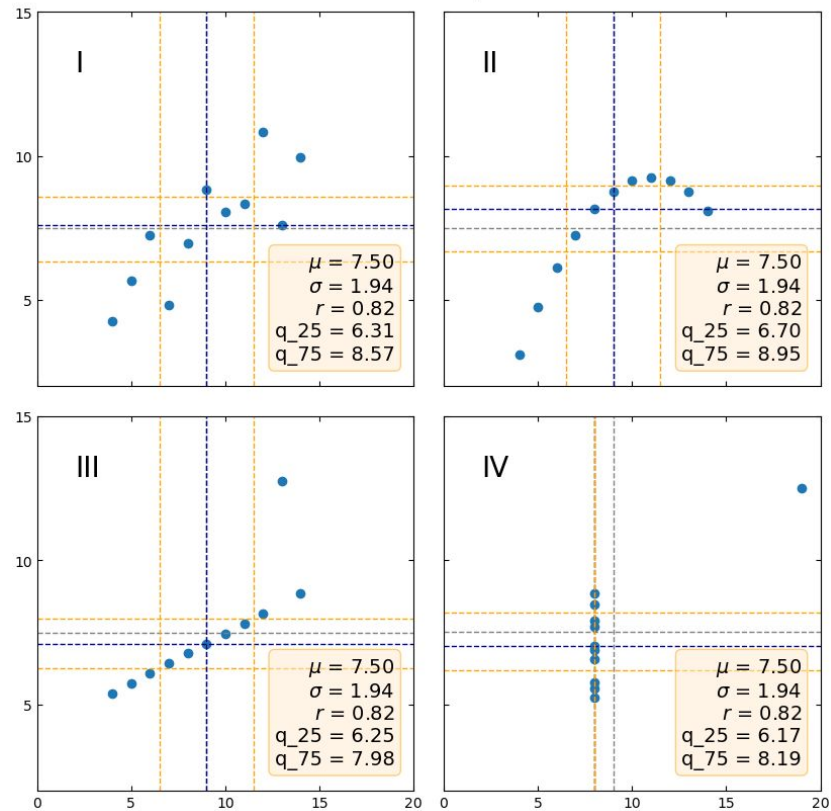
Anscombe's quartet



EDA: Lies, damned lies, and statistics

Which statistical indicators you'd use to describe these datasets in a paper?

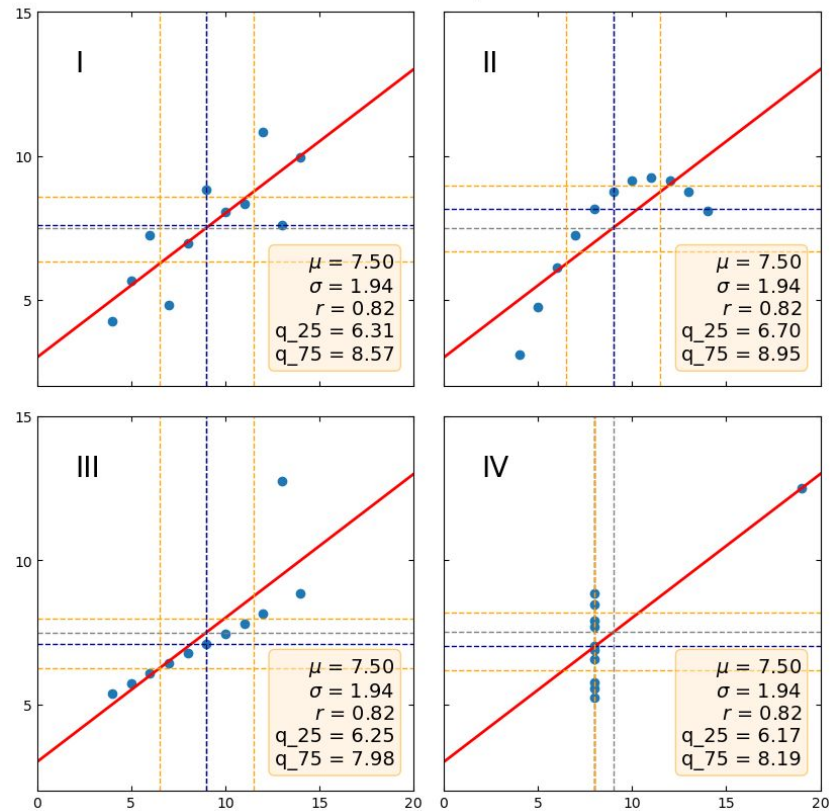
Anscombe's quartet



EDA: Lies, damned lies, and statistics

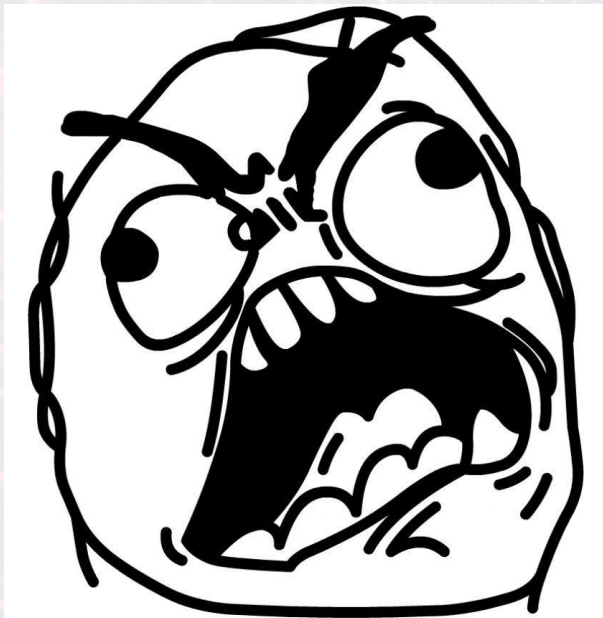
Which statistical indicators you'd use to describe these datasets in a paper?

Anscombe's quartet

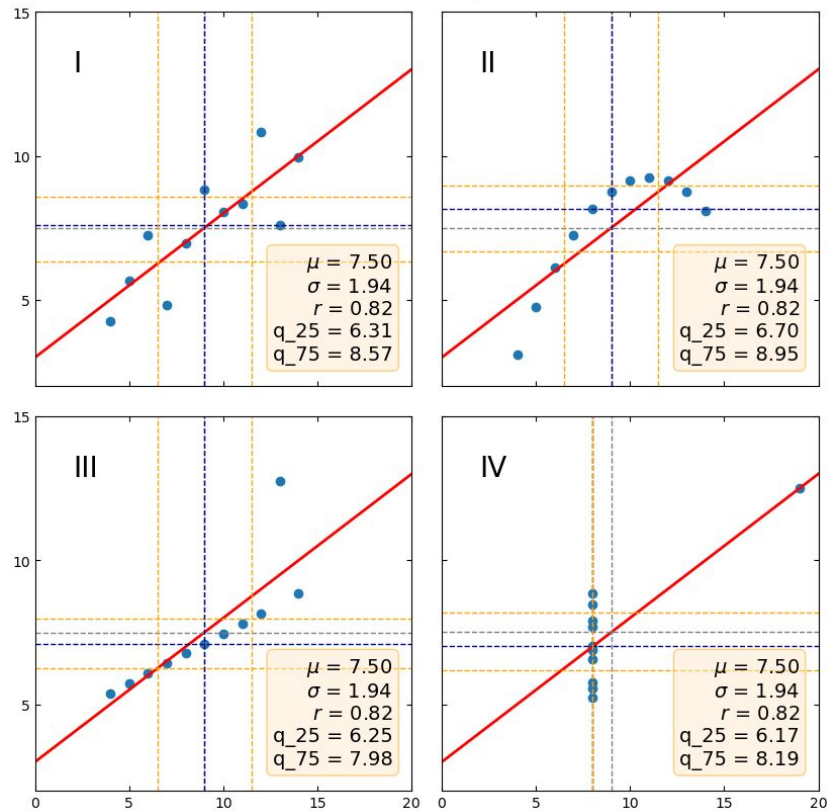


EDA: Lies, damned lies, and statistics

Which statistical indicators you'd use to describe these datasets in a paper?



Anscombe's quartet

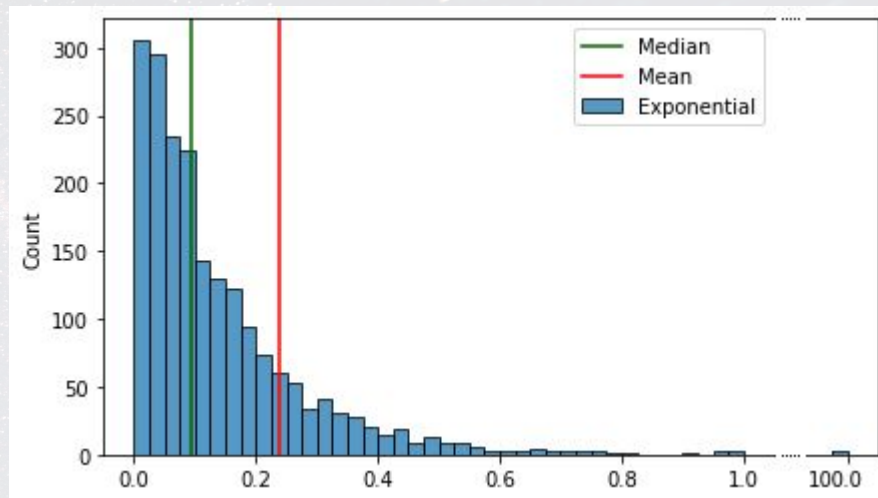
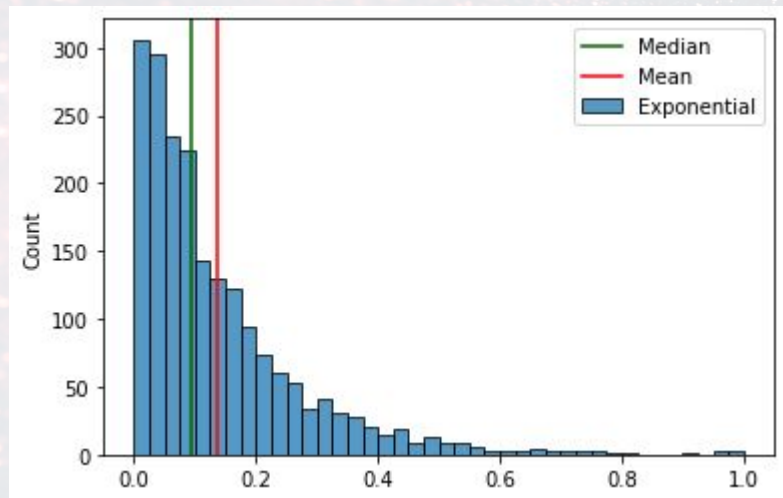


EDA: Distributions matter

Things to inspect:

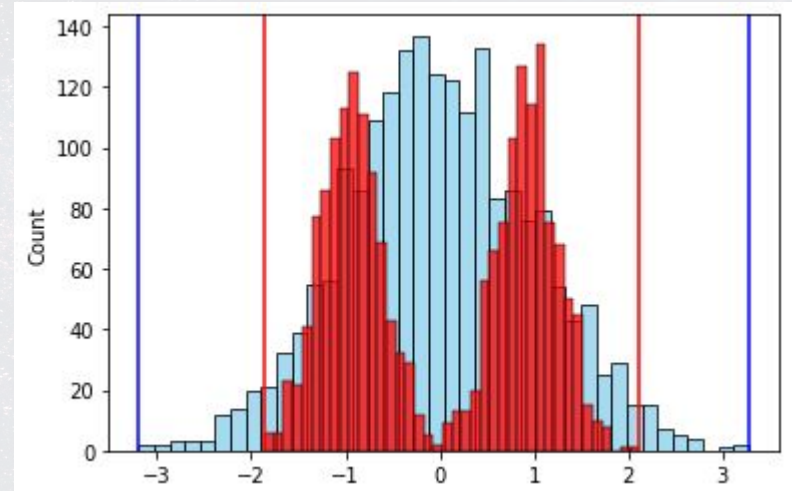
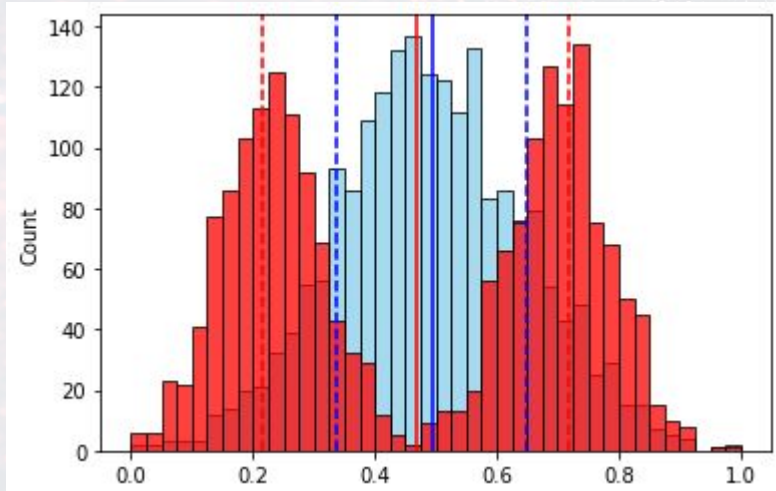
- Skewness (affects gradient-based optimization)
- Heavy tails (amplify variance)
- Multimodality (suggests hidden subpopulations)
- Outliers (distort scaling and correlation)

EDA: Exhibit A: 'Long tail'



<https://habr.com/ru/articles/527334/>

EDA: Exhibit B: Type of distribution and standatization



<https://habr.com/ru/articles/527334/>

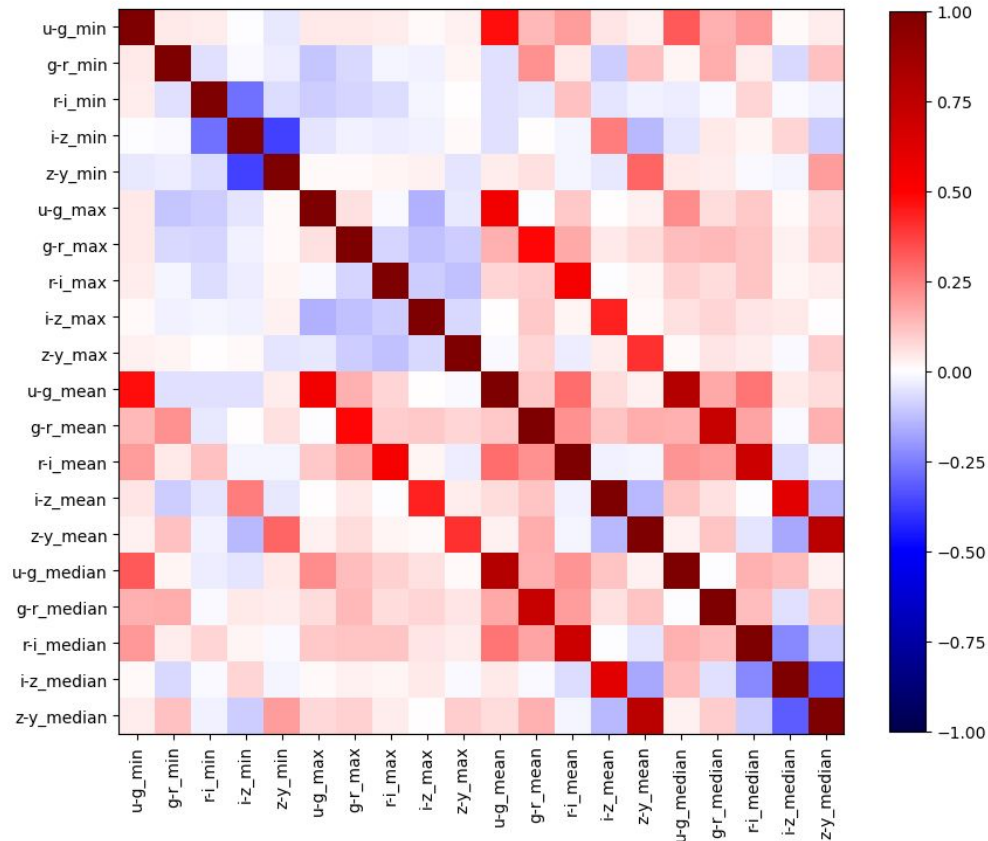
Pearson correlation:

$$\mathbf{r}_{XY} = \frac{\mathbf{cov}_{XY}}{\sigma_X \sigma_Y} = \frac{\sum (X - \bar{X})(Y - \bar{Y})}{\sqrt{\sum (X - \bar{X})^2 \sum (Y - \bar{Y})^2}}.$$

- Sensitive to outliers
- Misses nonlinear structure
- Does not imply causation!

High correlation → possible redundancy → risk of overfitting later, however,
conditional feature importance is a thing.

EDA: Correlation analysis



EDA: Label leakage detection

Label leakage – ‘use of information during model training that would not be available at prediction time’.

Examples:

- using total number of visits to a doctor when trying to predict illness recurrence risk
- using total number of observations when trying to classify rare, super-interesting objects
- timestamps, coordinates, hand-written labels on X-ray images...
- features extraction or any kind of preprocessing done separately for different classes of objects (with some parameters changing in the process)))

Leads to artificially high validation performance

If corr matrix shows super-high correlation of one or few features with the label - you're in trouble (likely)).

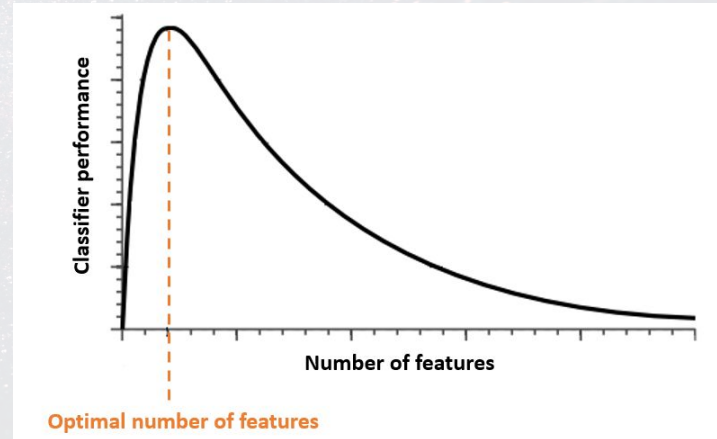
More ≠ better: Curse of Dimensionality

$$\lim_{d \rightarrow \infty} E \left(\frac{\text{dist}_{\max}(d) - \text{dist}_{\min}(d)}{\text{dist}_{\min}(d)} \right) \rightarrow 0$$

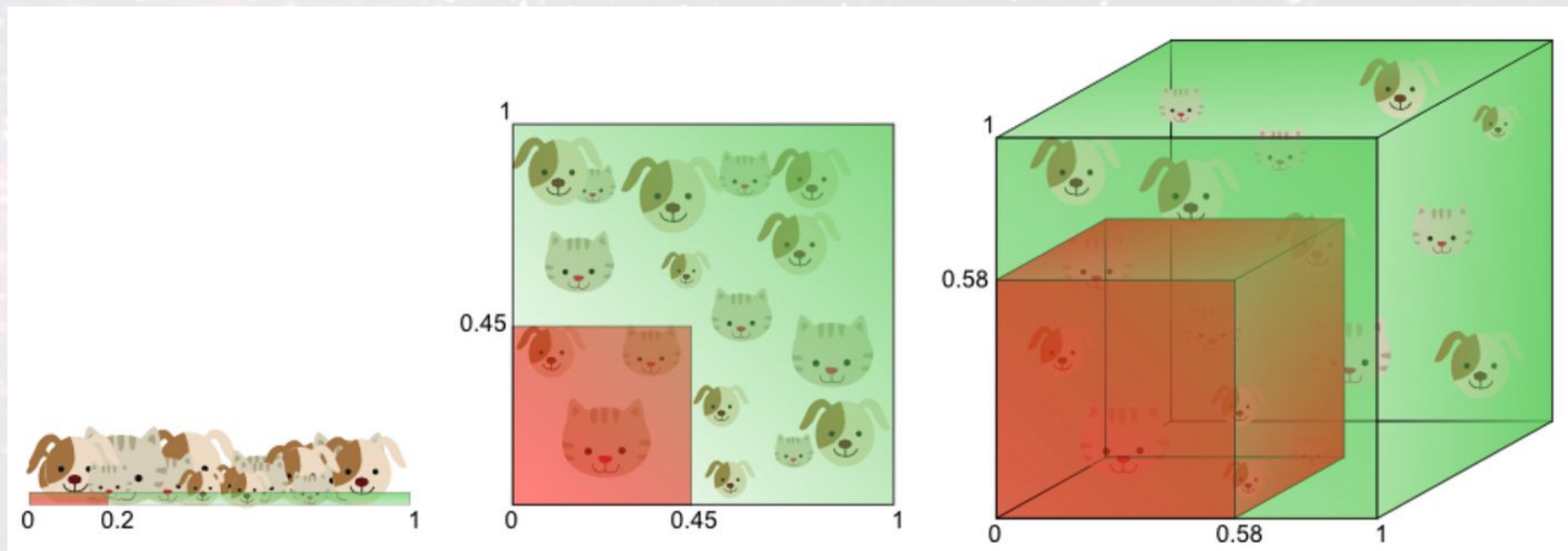
As feature dimension increases:

- Distances become similar
- Data becomes sparse
- Density estimation becomes unreliable

High-dimensional space is mostly empty :/



More $\neq$ better: Curse of Dimensionality



More ≠ better: Bias-Variance Tradeoff

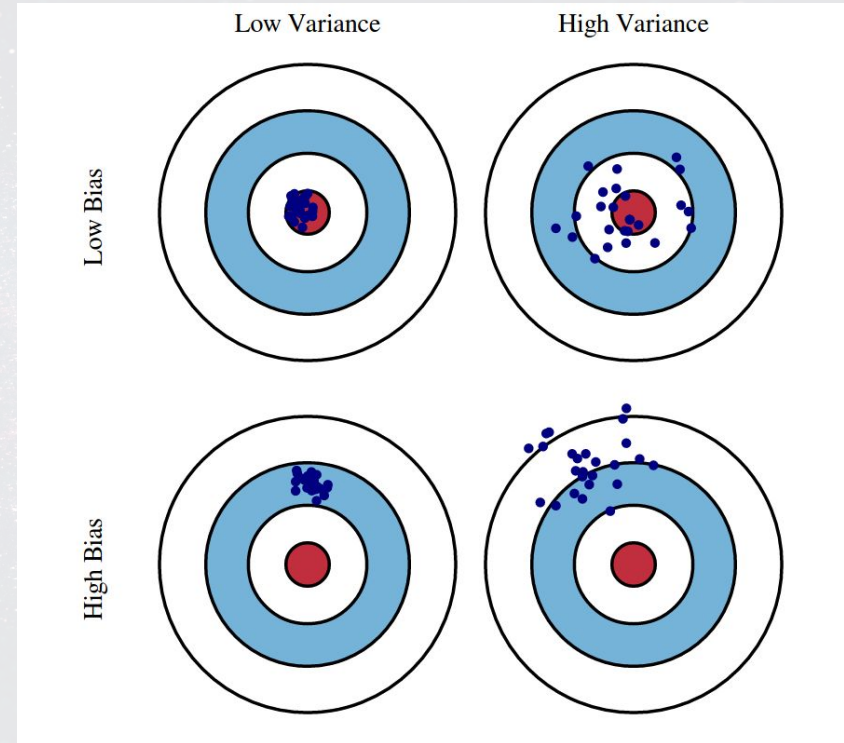
Small dataset + many features -> high variance.

Model complexity increases -> training error decreases. Because the model memorizes everything!

Validation error eventually increases (good performance on train split, but bad on unfamiliar data).

Consequence – **overfitting**: the model learns noise instead of signal.

Rule of thumbs: $\text{num_samples} > x * \text{num_features}$
(x commonly equals 5, but it depends)



Feature selection

Goal: Reduce variance, remove noise, and lower parameter space dimensionality.

Methods:

- Correlation filtering: remove highly correlated features
- Greedy selection: iterative addition/removal
- Tree-based importance: model-based ranking
- Permutation importance: measure performance drop when feature shuffled

Selection reduces overfitting risk.

The Holy Trinity: Train / Validation / Test

Classic split of the whole dataset with labels: train/validation/test, 60/20/20%.

Train: Fit parameters (the subset on which you train your models).

Validation: Tune hyperparameters (the subset you use for evaluating the performance while tweaking the knobs).

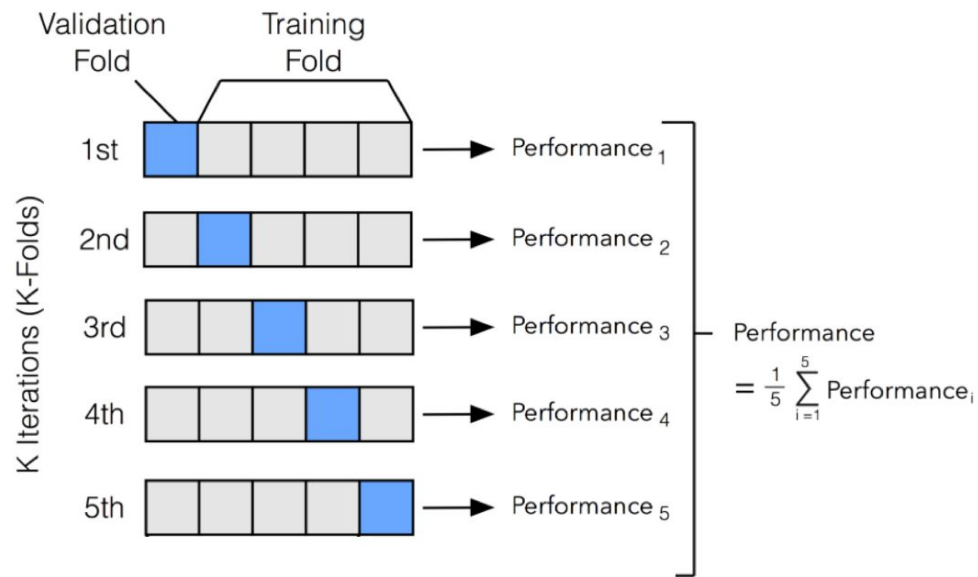
Test: Final unbiased estimate (the subset which numbers you report in the paper).

Preprocessing must be **learned** ('fit' methods) on training data only, then **applied** ('transform' methods) on everything else.

If you don't have that much data...

k-fold cross-validation:

- Split data into k parts
- Train on k-1
- Validate on remaining
- Average results
- Reduces variance of estimate.



Model Robustness and Distribution Shift

Model robustness: making sure that

- the model performs well if some feature is messed up
- if there is noise
- if feature relations have changed (data distribution shift).

Types of distribution shift:

- Covariate shift (features change)
- Concept drift (target relation changes)

ML models are **interpolators**, not **extrapolators**. Verify parameter space distributions!

Classification Metrics: TP, TN, FP, FN

For binary classification, there are following possibilities:

	Predicted class: 0	Predicted class: 1
Real class: 0	True negatives (TN)	False positives (FP)
Real class: 1	False negatives (FN)	True positives (TP)

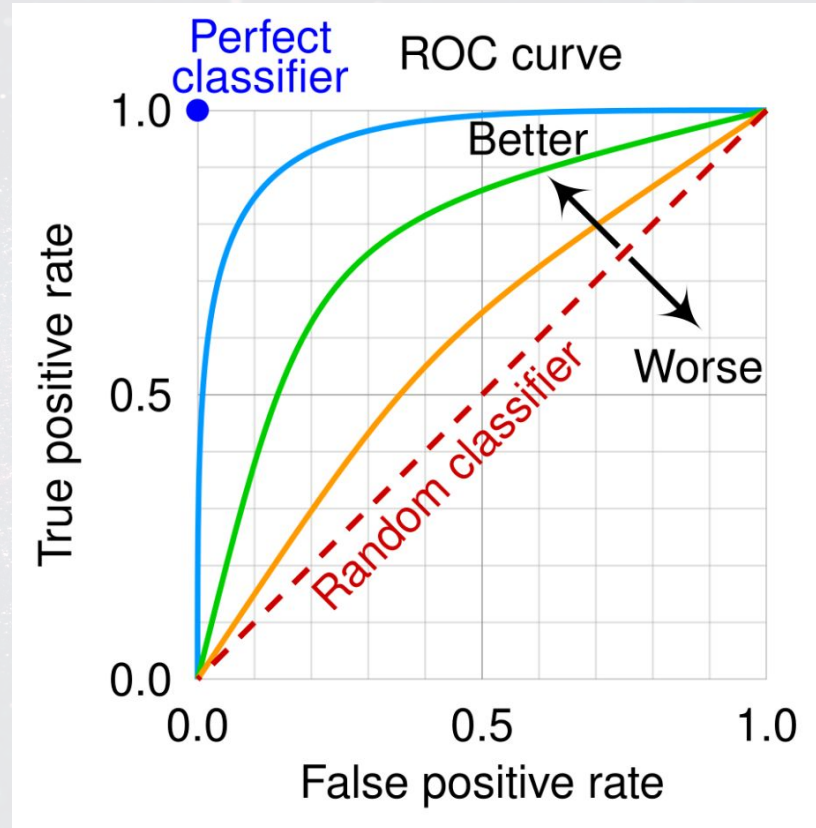
Classification Metrics: higher level

Precision = $TP / (TP + FP)$

Recall = $TP / (TP + FN)$

F1 = $2P * R / (P + R)$

Receiver operating characteristic curve (ROC):
True Positive Rate to False Positive Rate



Classification Metrics: higher level

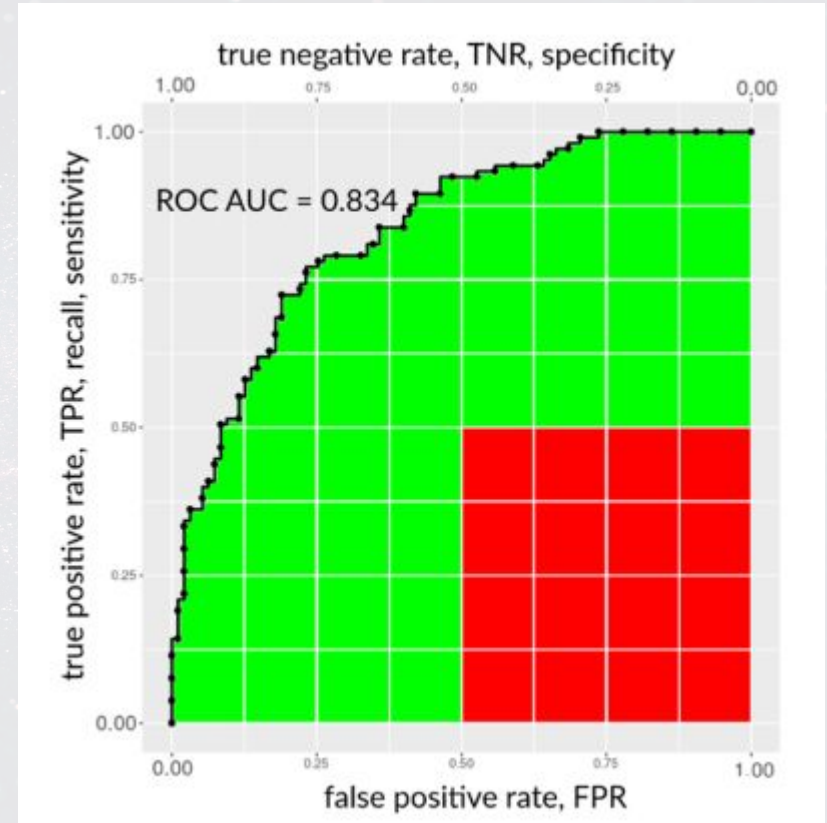
Precision = $TP / (TP + FP)$

Recall = $TP / (TP + FN)$

F1 = $2P * R / (P + R)$

Receiver operating characteristic curve (ROC):
True Positives to False Positives

Area Under the Curve: rough metric to compare
model performances



Classification Metrics: higher level

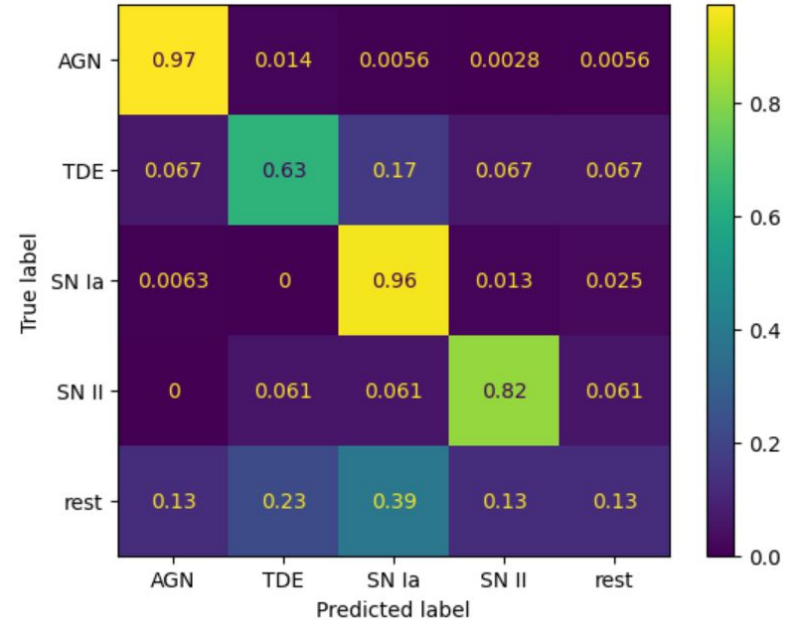
```
from sklearn.metrics import classification_report  
  
print(classification_report(y_test, pred))
```

	precision	recall	f1-score	support
0	0.98	0.97	0.98	357
1	0.58	0.63	0.60	30
2	0.88	0.96	0.92	158
3	0.75	0.82	0.78	33
4	0.29	0.13	0.18	31
accuracy			0.90	609
macro avg	0.69	0.70	0.69	609
weighted avg	0.89	0.90	0.89	609

Classification Metrics: higher level

```
from sklearn.metrics import ConfusionMatrixDisplay
```

```
ConfusionMatrixDisplay.from_predictions(y_test, pred,  
                                       display_labels=['AGN', 'TDE', 'SN Ia', 'SN II',  
                                                       'rest'], normalize='true')
```



Regression metrics: MAE and RMSE

Mean absolute error (MAE):

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Root mean square error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Difference between loss function and metric:

Loss is for the algorithm (has to be differentiable and computationally cheap), metric is for humans (easy to understand, relevant to the problem).

You can (and sometimes should!) write your own loss and metric functions!



Regression metrics: MAE and RMSE

Mean absolute error (MAE):

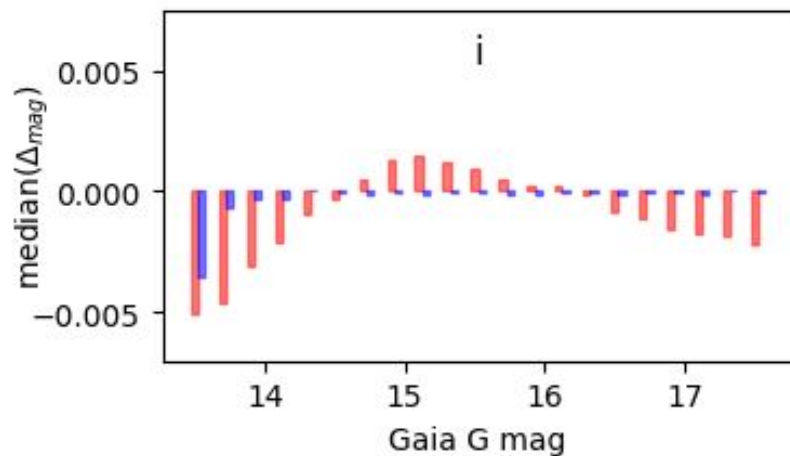
$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Root mean square error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

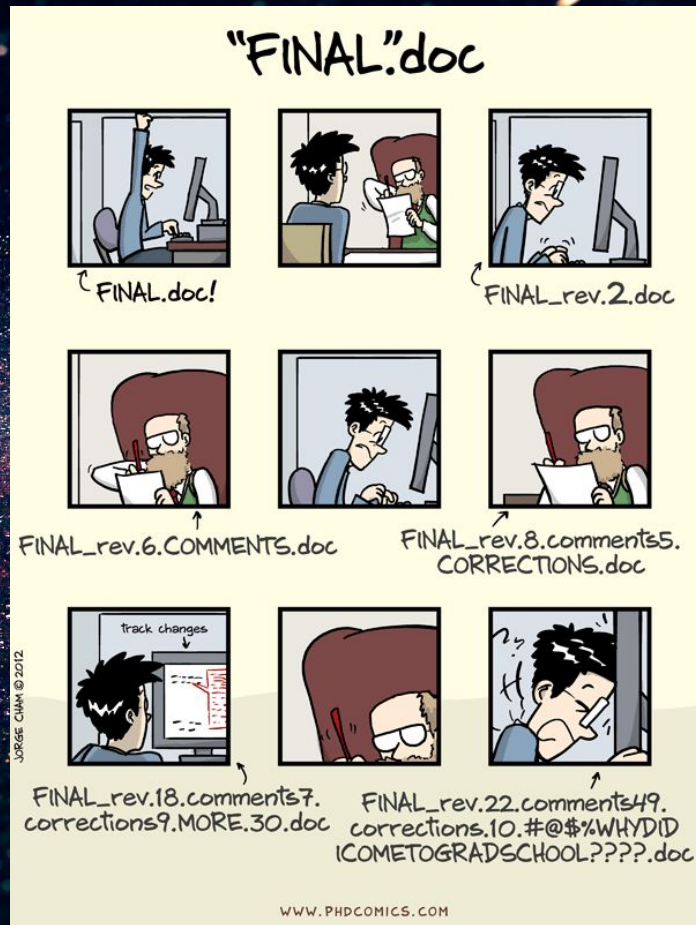
Remember: statistics knows how to lie. Single number is not informative.

Use residual plots and binned statistics (in 1D, 2D, nD...)



Final notes: Reproducibility

- Fix random seeds
- Save train/test splits
- Log preprocessing steps
- Version datasets (not just in file names :))
- Separate config from code
- Use git (no, seriously, use git, please)



Final notes: Workflow

Data

EDA -> Cleaning -> Feature engineering -> Split -> Train -> Evaluate (-> Rinse -> Repeat)

Final notes: Workflow

Data

EDA -> Cleaning -> Feature engineering -> Split -> Train -> Evaluate (-> Rinse -> Repeat)

*70% of the time
90% of success*

Further reading

Classical textbook: C. Bishop “Pattern Recognition and Machine Learning”

Neural networks specific: Goodfellow, Bengio, Courville, “Deep learning”

Down-to-earth, low-math textbook: Muller, “Introduction to Machine Learning with Python”